

Da Semana 07 à Semana 10

Conectando os fundamentos de CSS ao CSS Avançado — Flexbox, Especificidade e Responsividade

Desenvolvimento Front-End [React] T4 | Módulo 1 | Este material não traz conteúdo novo — ele mostra, ponto a ponto, como cada coisa que você já aprendeu na Semana 07 é a base literal do que vamos praticar a fundo esta semana. A ideia é simples: **nada aqui é assunto desconhecido, é o mesmo conhecimento crescendo.**

Como ler este material

Cada seção abaixo segue o mesmo formato: primeiro, **o que você já sabe** (com o código exatamente como apareceu na Semana 07); depois, **o que a Semana 10 acrescenta**; e por fim, uma comparação lado a lado entre a versão "básica" e a versão "avançada" do mesmo problema.

O fio condutor: o mesmo projeto, aprofundado

Você talvez não tenha notado, mas o exercício final da Semana 07 (HTML + CSS, construir a tela com variáveis de cor e fonte) já era a primeira versão da página inicial da **Bikcraft** — o mesmo projeto que você evolui desde então. A tabela abaixo mostra a linha do tempo completa.

Semana	O que a Bikcraft ganhou	Ferramenta usada
07	Nasce a home e a página de catálogo: estrutura HTML, cores/fontes em variáveis, Flexbox básico, Box Model	HTML, CSS, <code>display: flex</code>
08	<code>id</code> / <code>class</code> em cada bicicleta + captura dos dados via JavaScript	DOM, <code>getElementById</code> , <code>querySelectorAll</code>
09	Página de contato: formulário completo, captura no <code>submit</code> , confirmação na tela	Formulários HTML, JavaScript
10	A mesma página de contato ganha responsividade, layout em duas colunas e acabamento com bibliotecas	Assunto desta semana

CONEXÃO 1 — De "display: flex" ao Flexbox de produção

O que você já sabe (Semana 07)

Você já monta um flex container, já conhece os dois eixos, já usa `justify-content`, `align-items`, `flex-direction`, o atalho `flex` e o `gap`. É exatamente o menu vertical e as colunas que você construiu no exercício da aula:

```
.menu-vertical {
  display: flex;
  flex-direction: column; /* organiza os itens verticalmente */
  gap: 8px;
}

.flex-column {
  flex: 1; /* cresce e divide o espaço igualmente com os vizinhos */
  display: flex;
  justify-content: center;
  align-items: center;
}
```

O que a Semana 10 acrescenta

- **flex-wrap em layout real:** na Semana 07 você viu o conceito ("itens quebram linha"); agora ele resolve um problema de verdade — os cards da página de contato do Bikcraft, que precisam quebrar linha sozinhos em telas estreitas.
- **flex-direction responsivo:** a mesma propriedade que você usou para fixar `column` no menu vertical agora troca de valor dentro de uma `media query` — `column` no celular, `row` no desktop.
- **flex em proporções desiguais:** você já viu `flex: 2` vs `flex: 1` (a coluna A com o dobro do espaço). Na Semana 10, essa mesma ideia monta as duas colunas do contato: `flex: 5` e `flex: 7`.

Comparando o código: o mesmo Flexbox, antes e depois

SEMANA 07 — fixo

```
.container {
  display: flex;
}

.item-grande { flex: 2; }
.item-pequeno { flex: 1; }

/* sempre em linha, em
   qualquer tamanho de tela */
```

SEMANA 10 — responsivo

```
.secao-contato {
  display: flex;
  flex-direction: column; /* celular */
  gap: 40px;
}

@media (min-width: 64em) {
  .secao-contato { flex-direction: row; }
  .coluna-dados { flex: 5; }
  .coluna-formulario { flex: 7; }
}
```

O que realmente mudou?

Nada na essência do Flexbox — o que muda é que agora o valor de `flex-direction` depende do tamanho da tela, dentro de uma `@media`. É a mesma propriedade que você já sabia usar, só que agora "condicionada" à largura da tela.

CONEXÃO 2 — De "unidades de medida" à responsividade de verdade

O que você já sabe (Semana 07)

Você já viu a comparação entre `px`, `%`, `em`, `rem` e `vh / vw`, e já sabe a regra: `rem` é a preferida para fonte porque respeita a configuração de acessibilidade do usuário. Também já conhece o Box Model e o `box-sizing`.

Unidade	O que você aprendeu na S07
<code>px</code>	Valor absoluto — nunca muda, em nenhuma tela
<code>%</code>	Relativa ao elemento pai
<code>rem</code>	Relativa ao <code>font-size</code> do <code><html></code> — preferida para fontes/acessibilidade
<code>vh / vw</code>	Percentual da altura/largura da tela (viewport)

O que a Semana 10 acrescenta

- **A meta viewport**: a peça que faltava para essas unidades relativas (`%`, `vh`, `vw`) funcionarem de verdade no celular — sem ela, o navegador mobile simula uma tela larga e todo o cálculo de `% / vw` fica errado na prática.
- **Mobile-first + @media (min-width: ...)**: a estratégia de escrever o CSS base para o celular e usar media queries só para acrescentar o que telas maiores permitem.
- **clamp(mínimo, ideal, máximo)**: uma função nova, que combina exatamente as unidades que você já conhece — normalmente um `rem` (mínimo/máximo) com um `vw` (o ideal, que acompanha a tela).

Comparando o código: as mesmas unidades, agora combinadas

SEMANA 07 — isoladas

```
.titulo {
  font-size: 2rem; /* fixo,
    não acompanha a tela */
}
.banner {
  padding: 20px; /* fixo em px */
}
```

SEMANA 10 — combinadas

```
.hero-titulo {
  /* rem no mínimo/máximo,
    vw no meio do caminho */
  font-size: clamp(2.5rem, 6vw, 4rem);
}
@media (min-width: 64em) {
  .cabecalho { padding: 16px 100px; }
}
```

O que realmente mudou?

`rem` e `vw` continuam sendo exatamente o que eram na Semana 07. A novidade é `clamp()`, que usa as duas ao mesmo tempo — e a `media query`, que aplica uma regra só a partir de certo tamanho de tela.

CONEXÃO 3 — De "ID tem alta especificidade, use com moderação" ao cálculo completo

O que você já sabe (Semana 07)

Na revisão relâmpago de seletores, você já viu os três tipos e uma dica direta sobre cada um:

Seletor	O que você já ouviu na S07
Tipo (<code>p</code>)	"Atinge todos os elementos daquela tag. Baixa especificidade."
Classe (<code>.card</code>)	"Reutilizável em vários elementos. O mais usado no dia a dia."
ID (<code>#header</code>)	"Único na página. Alta especificidade — use com moderação. "

E, nas boas práticas de estilização, você já ouviu para "preferir classes reutilizáveis a IDs para estilo" e "evitar estilizar direto no HTML com o atributo `style`".

O que a Semana 10 acrescenta

A Semana 10 pega essas três frases — "baixa especificidade", "alta especificidade, use com moderação", "evite estilo inline" — e transforma cada uma em um **número**, para você parar de decidir "no feeling" e passar a calcular:

O que a S07 disse	O número que a S10 dá pra isso
Estilo inline (<code>style="..."</code>)	O mais forte de todos — por isso a S07 já dizia para evitar
ID — "alta especificidade"	Peso 100
Classe — "o mais usado no dia a dia"	Peso 10
Tipo/tag — "baixa especificidade"	Peso 1

Com esses números, quando duas regras miram o mesmo elemento, você soma os pesos de cada seletor e compara — a maior soma vence. Em caso de empate, vence a **cascata** (a última regra escrita no arquivo). E propriedades como `color` / `font-family` se propagam por **herança** do pai para o filho — mas `margin` / `padding`, não.

Comparando o código: da dica solta ao cálculo

SEMANA 07 — a dica

```
/* seletor de tipo */
p { color: #333; }      /* baixa especificidade */
```

SEMANA 10 — o cálculo

```
a { color: #fff; }      /* peso 1 (tag) */
.cabecalho a { color: #000; } /* peso 10+1=11 → vence
#logo { color: red; }      /* peso 100 → venceria
```

```
/* seletor de id */  
#header { height: 80px; } /* alta – use com moderação
```

```
/* nunca resolva empate com !important –  
resolva aumentando a especificidade certa */
```

O que realmente mudou?

Você já sabia QUE um ID é mais forte que uma classe. Agora você sabe QUANTO (100 contra 10) — e isso é o que permite prever, com certeza, qual regra vai vencer antes mesmo de testar no navegador.

CONEXÃO 4 — Das boas práticas isoladas a um sistema de decisão

Veja como cada aviso solto que você recebeu na Semana 07 ganha uma explicação técnica completa na Semana 10 — não são regras novas, são as mesmas regras, agora com o "porquê" por trás:

Aviso da Semana 07	A teoria que a Semana 10 dá pra ele
"Evite estilizar direto no HTML com o atributo <code>style</code> "	Estilo inline tem a maior especificidade de todas — vence quase tudo, dificultando qualquer ajuste futuro por CSS
"Prefira classes reutilizáveis a IDs para estilo"	Classe = peso 10; ID = peso 100. Usar ID para estilo deixa a especificidade alta demais, difícil de sobrescrever depois
"Abusar de <code>!important</code> para forçar um estilo" (na lista de "evite isso")	<code>!important</code> atropela a especificidade normal e cria uma escalada — só outro <code>!important</code> consegue vencer o anterior
"Use <code>rem</code> para fontes — respeita a configuração do usuário"	É a mesma unidade usada dentro do <code>clamp()</code> para criar tipografia fluida sem quebrar a acessibilidade
O uso do arquivo <code>.css</code> externo é preferido (separar responsabilidades)	É o que garante que a ordem dos <code><link></code> no <code><head></code> controle a cascata — por isso o CSS do Bikcraft sempre entra depois do Bootstrap

Checklist: você já está pronto para a Semana 10 se...

- ✓ Sabe abrir um flex container com `display: flex` e diferenciar `flex-direction: row` de `column`.
- ✓ Sabe o que `flex: 1` faz e como mudar a proporção entre itens com números diferentes.
- ✓ Sabe diferenciar `px` (absoluto) de `%` / `rem` / `vh` / `vw` (relativos).
- ✓ Entende o Box Model (`content` → `padding` → `border` → `margin`).
- ✓ Sabe que um seletor de `id` "pesa" mais que um de `classe`, que pesa mais que um de tag.
- ✓ Já ouviu que `!important` e estilo inline são práticas a evitar.

Se todos esses pontos fazem sentido, a Semana 10 não vai apresentar nada desconhecido — ela vai dar nome, número e ferramenta profissional para instintos que você já está desenvolvendo desde a Semana 07.